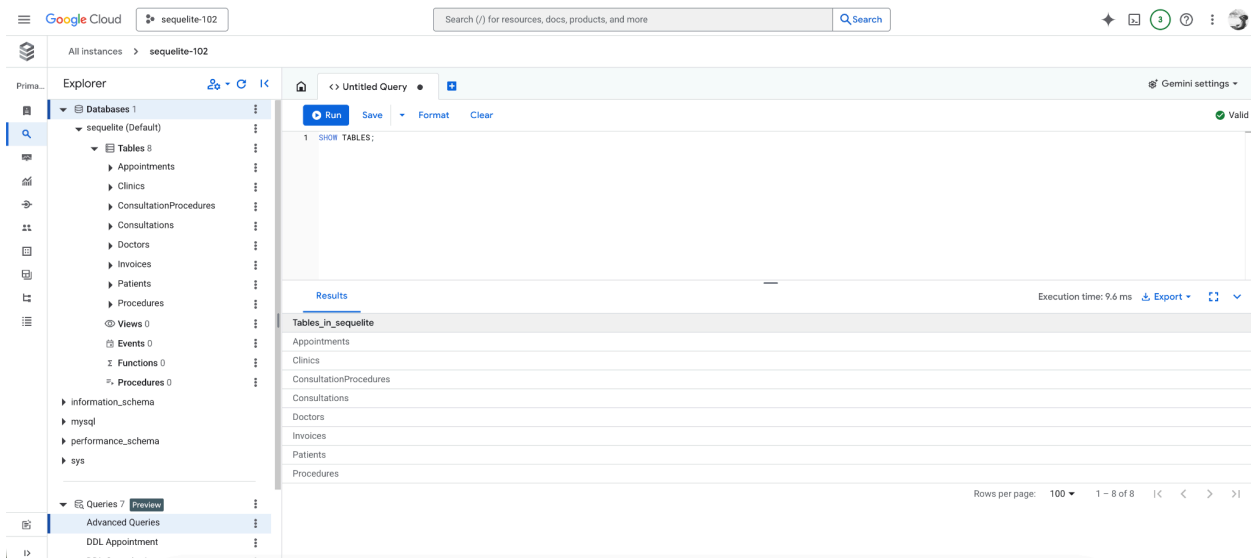


Digital Clinic Management System

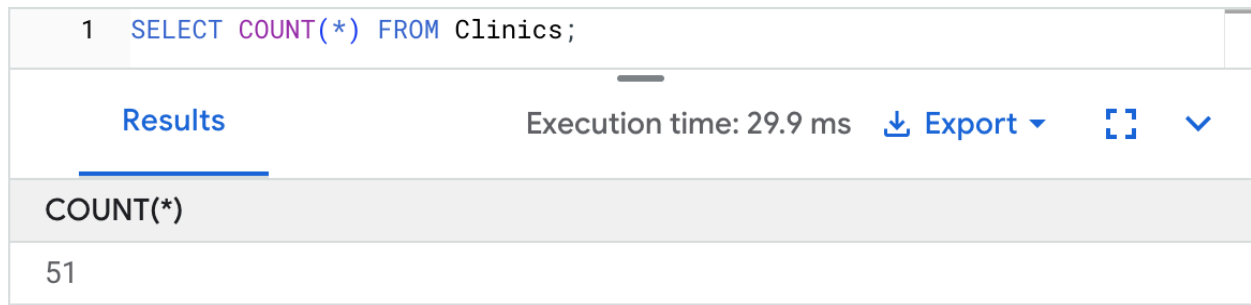
GCP Cloud SQL Implementation



DDL Commands

Clinics

```
CREATE TABLE Clinics(  
  ClinicID INT AUTO_INCREMENT PRIMARY KEY,  
  ClinicName VARCHAR(255) NOT NULL,  
  Zipcode CHAR(5) NOT NULL  
);
```



Doctors

```
CREATE TABLE Doctors(  
    DoctorID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(255) NOT NULL,  
    LastName VARCHAR(255),  
    EmailAddress VARCHAR(255) UNIQUE NOT NULL,  
    Specialization VARCHAR(255),  
    ConsultationFees DECIMAL(10,2),  
    Role VARCHAR(10) NOT NULL,  
    ClinicID INT,  
    CHECK (Role IN ('admin', 'consultant')),  
    FOREIGN KEY (ClinicID) REFERENCES Clinics(ClinicID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM Doctors;
Results Execution time: 11.0 ms Export SQL Close	
COUNT(*)	
1000	

Patients

```
CREATE TABLE Patients(  
    PatientID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(255) NOT NULL,  
    LastName VARCHAR(255),  
    EmailAddress VARCHAR(255),  
    Gender CHAR(1),  
    DateOfBirth DATE,  
    ClinicID INT,  
    CHECK (Gender IN ('M', 'F')),  
    FOREIGN KEY (ClinicID) REFERENCES Clinics(ClinicID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

```
1  SELECT COUNT(*) FROM Patients;
```

Results

Execution time: 31.4 ms

↓ Export ▾

⌂ ▾

COUNT(*)
1000

Appointments

```
CREATE TABLE Appointments(  
    AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
    AppointmentStatus CHAR(3) NOT NULL,  
    StartTime DATETIME NOT NULL,  
    EndTime DATETIME NOT NULL,  
    PatientID INT,  
    DoctorID INT,  
    CHECK (AppointmentStatus IN ('BKD','ACT','COM','CAN')),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE,  
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1 SELECT COUNT(*) FROM Appointments;	
Results	Execution time: 4.3 ms Export SQL v
COUNT(*)	
1784	

Consultations

```
CREATE TABLE Consultations(  
    ConsultationID INT AUTO_INCREMENT PRIMARY KEY,  
    AppointmentID INT,  
    HeartRate INT,  
    RespiratoryRate INT,  
    Temperature DECIMAL(4,1),  
    BloodOxygen DECIMAL(4,1),  
    SystolicBP INT,  
    DiastolicBP INT,  
    Weight DECIMAL(6,2),  
    Height DECIMAL(6,2),  
    ChiefComplaints TEXT,  
    Diagnosis TEXT,  
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM Consultations;
Results	Execution time: 8.1 ms Export Full Screen Close
COUNT(*)	
1784	

Procedures

```
CREATE TABLE Procedures(  
    ProcedureID INT AUTO_INCREMENT PRIMARY KEY,  
    ProcedureName VARCHAR(255) NOT NULL,  
    Amount DECIMAL(10,2) NOT NULL  
);
```

1	SELECT COUNT(*) FROM Procedures;
Results Execution time: 13.9 ms Export SQL Close	
COUNT(*)	
1001	

ConsultationProcedures

```
CREATE TABLE ConsultationProcedures(  
    ConsultationID INT,  
    ProcedureID INT,  
    PRIMARY KEY (ConsultationID, ProcedureID),  
    FOREIGN KEY (ConsultationID) REFERENCES Consultations(ConsultationID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,  
    FOREIGN KEY (ProcedureID) REFERENCES Procedures(ProcedureID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM ConsultationProcedures;
Results Execution time: 1.3 ms Export SQL Close	
COUNT(*)	
1000	

Invoices

```
CREATE TABLE Invoices(  
    InvoiceID INT AUTO_INCREMENT,  
    Amount DECIMAL(10, 2),  
    AppointmentID INT,  
    PRIMARY KEY (InvoiceID, AppointmentID),  
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM Invoices;	
Results	Execution time: 1.5 ms	↓ Export ↗ ✓
COUNT(*)		
1000		

Advanced SQL Queries

1. Get revenue for a clinic grouped by month, year

```
SELECT YEAR(EndTime) AS AccountYear, MONTH(EndTime) AS AccountMonth,
SUM(Amount)
FROM Appointments NATURAL JOIN Invoices NATURAL JOIN Doctors
WHERE AppointmentStatus = 'COM' AND ClinicID = 242
GROUP BY AccountYear, AccountMonth
ORDER BY AccountYear DESC, AccountMonth DESC;
```

Output (4 rows)

AccountYear	AccountMonth	SUM(Amount)
2022	6	3122623.00
2022	4	1606525.00
2021	1	1587342.00
2020	12	2281530.00

Indexing:

Before:

EXPLAIN
-> Sort: AccountYear DESC, AccountMonth DESC (actual time=65.1..65.1 rows=4 loops=1) -> Table scan on <temporary> (actual time=64.1..64.1 rows=4 loops=1) -> Aggregate using temporary table (actual time=64.1..64.1 rows=4 loops=1) -> Nested loop inner join (cost=97.3 rows=20.3) (actual time=54.5..63.9 rows=16 loops=1) -> Nested loop inner join (cost=74.9 rows=20.3) (actual time=12.7..20.9 rows=24 loops=1) -> Covering index lookup on Doctors using Doctors_ibfk_1 (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0564..0.108 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=1.45 rows=0.581) (actual time=0.592..0.594 rows=0.686 loops=35) -> Index lookup on Appointments using DoctorID (DoctorID=Doctors.DoctorID) (cost=1.45 rows=5.81) (actual time=0.59..0.591 rows=1.14 loops=35) -> Index lookup on Invoices using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=1 rows=1) (actual time=1.79..1.79 rows=0.667 loops=24)

First try: Index on AppointmentStatus

Why: The WHERE condition utilizes the AppointmentStatus = 'COM' condition for filtering. An index might make it more efficient.

EXPLAIN

-> Sort: AccountYear DESC, AccountMonth DESC (actual time=4.11..4.11 rows=4 loops=1) -> Table scan on <temporary> (actual time=4.08..4.08 rows=4 loops=1) -> Aggregate using temporary table (actual time=4.08..4.08 rows=4 loops=1) -> Nested loop inner join (cost=137 rows=103) (actual time=0.13..4 rows=16 loops=1) -> Nested loop inner join (cost=74.9 rows=103) (actual time=0.107..3.83 rows=24 loops=1) -> Covering index lookup on Doctors using Doctors_ibfk_1 (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0429..0.0664 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=1.46 rows=2.93) (actual time=0.107..0.107 rows=0.686 loops=35) -> Index lookup on Appointments using DoctorID (DoctorID=Doctors.DoctorID) (cost=1.46 rows=5.81) (actual time=0.106..0.106 rows=1.14 loops=35) -> Index lookup on Invoices using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.501 rows=1) (actual time=0.00562..0.0066 rows=0.667 loops=24)

Result: The following cost changes were observed:

- Cost before indexing: 97.3
- Cost after indexing: 137

Conclusion: MySQL did not use this index to drive the join. The index degraded the selectivity estimation. The number of rows estimated was 103 compared to the baseline 20 rows. This caused the cost to increase.

Second try: Composite index on AppointmentID, EndTime

Why: EndTime is being used in the GROUP BY in the form of YEAR() and MONTH() outputs.

Adding an index might make the input to the aggregation faster.

EXPLAIN

-> Sort: AccountYear DESC, AccountMonth DESC (actual time=4.78..4.78 rows=4 loops=1) -> Table scan on <temporary> (actual time=4.75..4.75 rows=4 loops=1) -> Aggregate using temporary table (actual time=4.75..4.75 rows=4 loops=1) -> Nested loop inner join (cost=248 rows=8.92) (actual time=0.274..2.96 rows=16 loops=1) -> Nested loop inner join (cost=243 rows=8.92) (actual time=0.246..2.77 rows=24 loops=1) -> Filter: ((Appointments.AppointmentStatus = 'COM') and (Appointments.DoctorID is not null)) (cost=180 rows=178) (actual time=0.106..1.18 rows=900 loops=1) -> Table scan on Appointments (cost=180 rows=1784) (actual time=0.0928..0.735 rows=1784 loops=1) -> Filter: (Doctors.ClinicID = 242) (cost=0.25 rows=0.05) (actual time=0.00165..0.00165 rows=0.0267 loops=900) -> Single-row index lookup on Doctors using PRIMARY (DoctorID=Appointments.DoctorID) (cost=0.25 rows=1) (actual time=0.00136..0.00139 rows=1 loops=900) -> Index lookup on Invoices using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.511 rows=1) (actual time=0.00639..0.00747 rows=0.667 loops=24)

Result: The following cost changes were observed:

- Cost before indexing: 97.3
- Cost after indexing: 248
- No change in aggregation costs

Conclusion: Since the GROUP BY condition is based on YEAR() and MONTH() functions and not the EndTime, the index cannot be directly used. The index is on the complete date-time value whereas the grouping is on computed expressions. It does not avoid the temporary table creation for aggregation.

2. Get patient visits for a clinic grouped by month, year

**SELECT YEAR(EndTime) AS VisitYear, MONTH(EndTime) AS VisitMonth,
COUNT(AppointmentID)**

FROM Appointments **NATURAL JOIN** Doctors
WHERE AppointmentStatus <> 'CAN' **AND** ClinicID = 242
GROUP BY VisitYear, VisitMonth
ORDER BY VisitYear **DESC**, VisitMonth **DESC**;

Output (9 rows)

VisitYear	VisitMonth	COUNT(AppointmentID)
2023	12	4
2023	9	4
2022	6	4
2022	4	4
2022	1	4
2021	5	4
2021	3	4
2021	1	8
2020	12	4

Indexing:

Before:

EXPLAIN
-> Sort: VisitYear DESC, VisitMonth DESC (actual time=19.4..19.4 rows=9 loops=1) -> Table scan on <temporary> (actual time=19.3..19.3 rows=9 loops=1) -> Aggregate using temporary table (actual time=19.3..19.3 rows=9 loops=1) -> Nested loop inner join (cost=74.9 rows=183) (actual time=5.36..12.1 rows=40 loops=1) -> Covering index lookup on Doctors using Doctors_ibfk_1 (ClinicID=242) (cost=3.76 rows=35) (actual time=1.18..1.21 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus <> 'CAN') (cost=1.47 rows=5.23) (actual time=0.308..0.311 rows=1.14 loops=35) -> Index lookup on Appointments using DoctorID (DoctorID=Doctors.DoctorID) (cost=1.47 rows=5.81) (actual time=0.307..0.309 rows=1.14 loops=35)

First try: Index on AppointmentStatus

Why: The WHERE condition utilizes the AppointmentStatus <> 'CAN' condition for filtering. An index might make it more efficient.

EXPLAIN
-> Sort: VisitYear DESC, VisitMonth DESC (actual time=3.04..3.04 rows=9 loops=1) -> Table scan on <temporary> (actual time=2.94..2.99 rows=9 loops=1) -> Aggregate using temporary table (actual time=2.94..2.94 rows=9 loops=1) -> Nested loop inner join (cost=74.9 rows=203) (actual time=0.379..0.746 rows=40 loops=1) -> Covering index lookup on Doctors using Doctors_ibfk_1 (ClinicID=242) (cost=3.76 rows=35) (actual time=0.279..0.312 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus <> 'CAN') (cost=1.47 rows=5.81) (actual time=0.00988..0.0116 rows=1.14 loops=35) -> Index lookup on Appointments using DoctorID (DoctorID=Doctors.DoctorID) (cost=1.47 rows=5.81) (actual time=0.00927..0.0105 rows=1.14 loops=35)

Result: The following cost changes were observed:

- Cost before indexing: 74.9
- Cost after indexing: 74.9

Conclusion: Since this is a negative condition, it is not very selective. Very few rows in the Appointments table have the CAN status, so the index still has to scan almost all the rows. Hence, MySQL preferred to ignore the index.

Second try: Composite index on AppointmentID, EndTime

Why: EndTime is being used in the GROUP BY in the form of YEAR() and MONTH() outputs. Adding an index might make the input to the aggregation faster.

EXPLAIN

```
-> Sort: VisitYear DESC, VisitMonth DESC (actual time=1.33..1.33 rows=9 loops=1) -> Table scan on <temporary> (actual time=1.31..1.31 rows=9 loops=1) -> Aggregate using temporary table (actual time=1.31..1.31 rows=9 loops=1) -> Nested loop inner join (cost=74.9 rows=183) (actual time=0.133..0.348 rows=40 loops=1) -> Covering index lookup on Doctors using Doctors_ibfk_1 (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0858..0.104 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus <> 'CAN') (cost=1.47 rows=5.23) (actual time=0.00536..0.00665 rows=1.14 loops=35) -> Index lookup on Appointments using DoctorID (DoctorID=Doctors.DoctorID) (cost=1.47 rows=5.81) (actual time=0.00509..0.00608 rows=1.14 loops=35)
```

Result: The following cost changes were observed:

- Cost before indexing: 74.9
- Cost after indexing: 74.9
- No change in aggregation costs

Conclusion: Since the GROUP BY condition is based on YEAR() and MONTH() functions and not the EndTime, the index cannot be directly used. The index is on the complete date-time value whereas the grouping is on computed expressions. It does not avoid the temporary table creation for aggregation.

3. Get total invoice amount for a particular appointment

```
SELECT SUM(OrderItemAmounts) AS InvoiceAmount
FROM (
  (SELECT Amount AS OrderItemAmounts
   FROM ConsultationProcedures NATURAL JOIN Procedures
   WHERE ConsultationID = 425)
  UNION ALL
  (SELECT ConsultationFees AS OrderItemAmounts
   FROM Doctors
   WHERE DoctorID = 170)
```

) AS OrderItems;

Output (1 row)

InvoiceAmount
1142.57

Indexing:

Before:

EXPLAIN
-> Aggregate: sum(OrderItems.OrderItemAmounts) (cost=3.62..3.62 rows=1) (actual time=0.177..0.177 rows=1 loops=1) -> Table scan on OrderItems (cost=2.16..3.42 rows=2) (actual time=0.169..0.17 rows=2 loops=1) -> Union all materialize (cost=0.9..0.9 rows=2) (actual time=0.167..0.167 rows=2 loops=1) -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.132..0.137 rows=1 loops=1) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=425) (cost=0.35 rows=1) (actual time=0.0373..0.0422 rows=1 loops=1) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.35 rows=1) (actual time=0.0933..0.0934 rows=1 loops=1) -> Rows fetched before execution (cost=0..0 rows=1) (actual time=116e-6..168e-6 rows=1 loops=1)

First try: Composite index on **DoctorID**, **ConsultationFees** on **Doctors** table

Why: Since ConsultationFees is attached to a particular doctor, adding this index might make finding the consultation fees for a doctor efficient.

EXPLAIN
-> Aggregate: sum(OrderItems.OrderItemAmounts) (cost=3.62..3.62 rows=1) (actual time=0.177..0.177 rows=1 loops=1) -> Table scan on OrderItems (cost=2.16..3.42 rows=2) (actual time=0.169..0.17 rows=2 loops=1) -> Union all materialize (cost=0.9..0.9 rows=2) (actual time=0.167..0.167 rows=2 loops=1) -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.132..0.137 rows=1 loops=1) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=425) (cost=0.35 rows=1) (actual time=0.0373..0.0422 rows=1 loops=1) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.35 rows=1) (actual time=0.0933..0.0934 rows=1 loops=1) -> Rows fetched before execution (cost=0..0 rows=1) (actual time=116e-6..168e-6 rows=1 loops=1)

Result: No change in costs after indexing, since DoctorID was already indexed.

4. Get most performed procedures for a doctor

```
SELECT ProcedureName, COUNT(ProcedureID) AS TimesPerformed
FROM ConsultationProcedures NATURAL JOIN Procedures NATURAL JOIN Consultations
NATURAL JOIN Appointments
WHERE AppointmentStatus = 'COM' AND DoctorID = 170
GROUP BY ProcedureName
HAVING COUNT(ProcedureID) > 0
ORDER BY TimesPerformed DESC;
```

Output (8 rows)

ProcedureName	TimesPerformed
Polypharmacy management	1
Interventional radiology procedures	1
Sedation for minor procedures	1
Plastic surgery	1
Cochlear implant surgery	1
Emergency surgical procedures	1
Comprehensive geriatric assessment	1
Allergy testing	1

Indexing:

Before:

EXPLAIN	
-> Sort: TimesPerformed DESC (actual time=53.7..53.7 rows=8 loops=1) -> Filter: ('count(ConsultationProcedures.ProcedureID)' > 0) (actual time=52.5..52.5 rows=8 loops=1) -> Table scan on <temporary> (actual time=51.4..51.4 rows=8 loops=1) -> Aggregate using temporary table (actual time=51.4..51.4 rows=8 loops=1) -> Nested loop inner join (cost=8.6 rows=1.6) (actual time=20.9..51.2 rows=8 loops=1) -> Nested loop inner join (cost=7.08 rows=1.6) (actual time=20.8..23.3 rows=8 loops=1) -> Nested loop inner join (cost=5.92 rows=1.6) (actual time=20.8..22.1 rows=12 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=4.16 rows=1.6) (actual time=0.138..0.199 rows=12 loops=1) -> Index lookup on Appointments using DoctorID (DoctorID=170) (cost=4.16 rows=16) (actual time=0.125..0.145 rows=16 loops=1) -> Covering index lookup on Consultations using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=1.06 rows=1) (actual time=1.82..1.82 rows=1 loops=12) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=Consultations.ConsultationID) (cost=0.687 rows=1) (actual time=0.0961..0.0998 rows=0.667 loops=12) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.912 rows=1) (actual time=3.49..3.49 rows=1 loops=8)	^

First try: Index on ProcedureName

Why: ProcedureName is used for grouping in the GROUP BY. Indexing this attribute might make the aggregation more efficient.

EXPLAIN	
-> Sort: TimesPerformed DESC (actual time=2.38..2.38 rows=8 loops=1) -> Filter: ('count(ConsultationProcedures.ProcedureID)' > 0) (actual time=2.35..2.36 rows=8 loops=1) -> Table scan on <temporary> (actual time=2.35..2.35 rows=8 loops=1) -> Aggregate using temporary table (actual time=2.34..2.34 rows=8 loops=1) -> Nested loop inner join (cost=5.84 rows=1.6) (actual time=0.223..0.431 rows=8 loops=1) -> Nested loop inner join (cost=5.28 rows=1.6) (actual time=0.192..0.354 rows=8 loops=1) -> Nested loop inner join (cost=4.72 rows=1.6) (actual time=0.176..0.269 rows=12 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=4.16 rows=1.6) (actual time=0.149..0.175 rows=12 loops=1) -> Index lookup on Appointments using DoctorID (DoctorID=170) (cost=4.16 rows=16) (actual time=0.136..0.146 rows=16 loops=1) -> Covering index lookup on Consultations using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.312 rows=1) (actual time=0.0066..0.00747 rows=1 loops=12) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=Consultations.ConsultationID) (cost=0.312 rows=1) (actual time=0.00445..0.00676 rows=0.667 loops=12) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.312 rows=1) (actual time=0.00781..0.00791 rows=1 loops=8)	^

Result: The following cost changes were observed:

- Top nested loop cost (before): 8.6; Top nested loop cost (after): 5.84;
- Middle nested loop cost (before): 7.08; Middle nested loop cost (after): 5.28;
- Bottom nested loop cost (before): 5.92; Bottom nested loop cost (after): 4.72;

Conclusion: Adding an index on the ProcedureName reduced costs for aggregation and nested loop joins significantly. The index can be used in grouping leading to less internal sorting and fewer temporary table estimates, and as a result corresponding join costs decrease as well.

Final Indexing

The final indexes (apart from primary keys) added after analyzing the advanced queries and other use cases are as follows:

- **AppointmentStatus** on **Appointments** table (*Fetching appointments by status on dashboard*)
- **(StartTime, EndTime)** on **Appointments** table (*For fetching appointments by date on dashboard and patient profile*)
- **(FirstName, LastName)** on **Patients** table (*For faster patient search*)
- **ProcedureName** on **Procedures** table (*Advanced query 4 and for faster procedure search by name during consultation*)